

Memento Design Pattern

Intent

Capture an object's internal state so it can be **saved and later restored** — without exposing that object's internals to the outside world. In one word: **undo**. You snapshot an object now, keep the snapshot somewhere, and roll the object back to it later, all while the object's private data stays private.

Here the object is a **Document** you keep writing to; each snapshot is a **DocumentMemento** ; and a **History** keeps the snapshots so you can restore an earlier version.

UML class diagram



The three roles (this pattern is defined by them)

originator/Document	the ORIGINATOR	– the object whose state we snapshot & restore
memento/DocumentMemento	the MEMENTO	– an immutable snapshot of that state
caretaker/History	the CARETAKER	– stores mementos, but never looks inside them
MementoDesignPattern	the demo	– write, snapshot, write more, then roll back

- **Originator** — creates a memento of its own state and knows how to restore itself from one.
- **Memento** — a passive value object holding the saved state. It's the only thing that crosses between originator and caretaker.
- **Caretaker** — holds mementos for safe-keeping (the history/undo stack) but treats them as **opaque** — it never reads or edits the state inside.

The code, line by line

Document — the originator

```
public class Document {
    private String content;

    public Document(String content) { this.content = content; }

    public void write(String text) { this.content += text; }

    public String getContent() { return this.content; }

    public DocumentMemento createMemento() {
        return new DocumentMemento(this.content);
    }

    public void restoreFromMemento(DocumentMemento documentMemento) {
        this.content = documentMemento.getSavedContent();
    }
}
```

- **private String content** — the state we care about protecting and snapshotting. It's private; nobody edits it except through `Document`'s own methods.
- **write(text)** — mutates the state (appends text). This is the "normal work" that we might later want to undo.
- **createMemento()** — the **save** operation. The document packages a copy of its *current* `content` into a new `DocumentMemento` and hands it out. Note the originator is the one that *builds* the memento — it decides what goes in, so it controls exactly which parts of its state are captured.

- `restoreFromMemento(m)` — the **undo** operation. The document reads the saved content back out of a memento and overwrites its own `content` with it. Again the originator is the one **reading** the memento — restoring is its responsibility, not the caretaker's.

DocumentMemento — the memento

```
public class DocumentMemento {
    private String content;

    public DocumentMemento(String content) { this.content = content; }

    public String getSavedContent() { return this.content; }
}
```

- A tiny, **passive** value object: a constructor that captures the state and a getter to read it back. It has no logic — it's just a labeled snapshot.
- It stores a `String`, which is **immutable** in Java, so the snapshot is a true frozen copy: later `write(...)` calls on the document cannot alter what's already inside a memento. (If the state were a **mutable** object, the memento would need to store a **defensive copy** — see **Why immutability matters** below.)

History — the caretaker

```
public class History {
    private List<DocumentMemento> mementos;

    public History() { this.mementos = new ArrayList<>(); }

    public void addMemento(DocumentMemento documentMemento) { this.mementos.add(documentMemento); }

    public DocumentMemento getMemento(int index) { return this.mementos.get(index); }
}
```

- Holds a **list of mementos** — this is the undo history / version list.
- `addMemento(...)` pushes a new snapshot; `getMemento(index)` retrieves one to restore later.
- ****What it deliberately does *not* do:**** it never calls anything that interprets or changes the snapshot's contents. It stores and returns `DocumentMemento` objects whole. The caretaker is a custodian, not an editor — it moves snapshots around without ever knowing what's in them.

MementoDesignPattern — the demo

```
Document document = new Document("Initial content\n");
History history = new History();

document.write("Additional content\n");
history.addMemento(document.createMemento());    // snapshot #0

document.write("More content\n");
history.addMemento(document.createMemento());    // snapshot #1

document.restoreFromMemento(history.getMemento(0)); // roll back to snapshot #0

System.out.println(document.getContent());
```

- Start with `"Initial content\n"`.
- Write more, then **save** snapshot #0 (`Initial + Additional`).
- Write more, then **save** snapshot #1 (`Initial + Additional + More`).
- **Restore** snapshot #0 — the document rolls back, discarding `"More content"`.

Console output:

```
Memento Design Pattern
Initial content
Additional content
```

The `"More content"` line is gone because we rolled the document back to the state captured **before** it was written.

Why the design decisions

Why not just let callers save and set `content` directly?

Because that breaks encapsulation. If undo were done by reading and writing a public `content` field from outside, then every caller would depend on the document's internal representation — change `content` from a `String` to a rope/gap-buffer and all that code breaks. Memento keeps the save/restore *mechanism* while keeping the *state* private: the outside world holds a `DocumentMemento` it can't meaningfully open, and only the `Document` knows how to fill and read it.

Why three separate roles instead of one class doing everything?

Separation of concerns, and it's the whole structure of the pattern:

- The **originator** owns *what* state to save and *how* to restore — the only class that understands the content.
- The **caretaker** owns *when* to save and *which* snapshot to restore — the undo policy — without needing to understand the content at all.
- The **memento** is the neutral courier between them.

This means you can change the undo policy (single undo, multi-level, redo, branching history) entirely inside the caretaker without touching the document, and change the document's internals without touching the caretaker.

Why does the caretaker treat the memento as opaque?

So the snapshot can't be corrupted while it's in storage. If the caretaker could reach into a memento and change its content, a "saved" state could silently drift, and undo would restore something that was never actually the document's state. Keeping the memento opaque to the caretaker guarantees a snapshot is exactly what the originator put in it. (In an ideal Java implementation this is enforced — see the note below on the "wide vs. narrow interface.")

Why does immutability matter here?

The memento must be a **frozen** copy of the state at save time. `String` is immutable, so storing the reference is enough — nothing can change it afterward. But if `Document`'s state were a `StringBuilder`, a `List`, or any mutable object, then `createMemento()` storing the *reference* would be a bug: later mutations to the live object would also mutate the "snapshot," so undo would restore the *current* state, not the saved one. The fix in that case is a **defensive copy** inside `createMemento()` (e.g. `new StringBuilder(content)`, `new ArrayList<>(list)`). Immutability is what makes the current code safe without a copy.

Why does the originator build *and* read the memento (not the caretaker)?

Because only the originator understands the state. Letting the originator create the memento means it controls exactly what is captured; letting it do the restore means it controls how the state is put back. The caretaker stays blissfully ignorant, which is exactly what keeps the document's internals encapsulated.

Execution flow (the demo)

```
main
├── new Document("Initial content\n")      content = "Initial content\n"
├── new History()                          mementos = []
├── document.write("Additional content\n")  content = "Initial\nAdditional\n"
├── history.addMemento(document.createMemento())
│   └── DocumentMemento("Initial\nAdditional\n")  mementos = [ #0 ]
├── document.write("More content\n")        content = "Initial\nAdditional\nMore\n"
├── history.addMemento(document.createMemento())
│   └── DocumentMemento("Initial\nAdditional\nMore\n")  mementos = [ #0, #1 ]
├── document.restoreFromMemento(history.getMemento(0))
│   └── content = #0.getSavedContent() = "Initial\nAdditional\n" ← rolled back
└── println(document.getContent())
    └── "Initial content\nAdditional content\n" (the "More content" line is undone)
```

Notice snapshot **#1** is still sitting in `history` — so you could redo/re-restore to it later with `getMemento(1)`. The history keeps every saved version.

Notes / possible improvements (not changed in the code)

- **Encapsulation could be tighter (the "wide vs. narrow interface")**. Here `DocumentMemento.getSavedContent()` is `public`, so the caretaker *could* read the saved content even though it doesn't. The textbook Java technique is to make the memento a **nested class** of the originator (or expose only a marker/narrow interface to the caretaker) so that **only** the originator can read the state, while the caretaker sees an opaque token. As written, the caretaker simply *chooses* not to look — the discipline is by convention, not enforced.
- **Mutable state would need a defensive copy** in `createMemento()`, as explained above. Safe here only because `String` is immutable.

- **Memory cost.** Each memento holds a full copy of the state; a long history of large snapshots costs memory. Real editors mitigate this with diffs/incremental mementos or a bounded history.
 - **Plain main, no Spring** — the pattern is pure OO structure and needs no container.
-

Relationship to the other Behavioral patterns

- **Memento (this module)** — externalize and later restore an object's state for undo, keeping the state encapsulated.
- **Command** — often paired with Memento: a command executes an action and stores a memento so it can `undo()` back to the prior state.
- **Prototype** (creational) — also copies an object, but to produce a new usable instance, not to stash-and-restore state.
- **State** — represents an object's current mode as an object; Memento snapshots state to *rewind* it rather than to switch behavior.

Reach for Memento when you need **undo/rollback, checkpoints, or history** and you want to snapshot an object's state without breaking its encapsulation.